

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №1
По дисциплине «Интеллектуальный анализ данных»
Тема: «РСА»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Шоева Е.Т.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться применять метод PCA для осуществления визуализации данных.

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование `numpy.linalg.eig` для вычисления собственных значений и собственных векторов и 2. с помощью `sklearn.decomposition.PCA` для непосредственного применения метода PCA – два независимых варианта решения);

2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;

3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;

4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на `github`.

Задание по вариантам

№ варианта	Выборка	Класс
11	seeds.zip	Последняя колонка

Ход работы

```
import zipfile
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

with zipfile.ZipFile('seeds.zip', 'r') as z:
    z.extractall('seeds_data')

columns = ['area', 'perimeter', 'compactness', 'length', 'width', 'asymmetry',
'groove_length', 'class']
df = pd.read_csv('seeds_data/seeds_dataset.txt', sep=r'\s+', names=columns,
header=None)

print("Пропущенные значения по столбцам до обработки:")
print(df.isna().sum())

df = df.fillna(df.mean(numeric_only=True))

X = df.drop(columns=['class']).values
y = df['class'].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

cov_matrix = np.cov(X_scaled.T)
eig_values, eig_vectors = np.linalg.eigh(cov_matrix)
```

```

idx = np.argsort(eig_values)[::-1]
eig_values = eig_values[idx]
eig_vectors = eig_vectors[:, idx]

W2 = eig_vectors[:, :2]
W3 = eig_vectors[:, :3]

X_pca2_manual = X_scaled.dot(W2)
X_pca3_manual = X_scaled.dot(W3)

colors = {1: 'r', 2: 'g', 3: 'b'}
labels_map = {1: 'Class 1', 2: 'Class 2', 3: 'Class 3'}

fig = plt.figure(figsize=(16, 6))

ax1 = fig.add_subplot(1, 2, 1)
for c in np.unique(y):
    mask = y == c
    ax1.scatter(X_pca2_manual[mask, 0], X_pca2_manual[mask, 1], c=colors[c],
label=labels_map[c])
ax1.set_xlabel('PC1')
ax1.set_ylabel('PC2')
ax1.set_title('Manual PCA (2D)')
ax1.legend()

ax2 = fig.add_subplot(1, 2, 2, projection='3d')
for c in np.unique(y):
    mask = y == c
    ax2.scatter(X_pca3_manual[mask, 0], X_pca3_manual[mask, 1],
X_pca3_manual[mask, 2], c=colors[c], label=labels_map[c])
ax2.set_xlabel('PC1')
ax2.set_ylabel('PC2')
ax2.set_zlabel('PC3')
ax2.set_title('Manual PCA (3D)')
ax2.legend()

plt.tight_layout()
plt.savefig('manual_pca.png')
plt.show()

pca2 = PCA(n_components=2)
X_pca2_sklearn = pca2.fit_transform(X_scaled)

pca3 = PCA(n_components=3)
X_pca3_sklearn = pca3.fit_transform(X_scaled)

fig = plt.figure(figsize=(16, 6))

```

```

ax1 = fig.add_subplot(1, 2, 1)
for c in np.unique(y):
    mask = y == c
    ax1.scatter(X_pca2_sklearn[mask, 0], X_pca2_sklearn[mask, 1], c=colors[c],
label=labels_map[c])
ax1.set_xlabel('PC1')
ax1.set_ylabel('PC2')
ax1.set_title('sklearn PCA (2D)')
ax1.legend()

ax2 = fig.add_subplot(1, 2, 2, projection='3d')
for c in np.unique(y):
    mask = y == c
    ax2.scatter(X_pca3_sklearn[mask, 0], X_pca3_sklearn[mask, 1],
X_pca3_sklearn[mask, 2], c=colors[c], label=labels_map[c])
ax2.set_xlabel('PC1')
ax2.set_ylabel('PC2')
ax2.set_zlabel('PC3')
ax2.set_title('sklearn PCA (3D)')
ax2.legend()

plt.tight_layout()
plt.savefig('sklearn_pca.png')
plt.show()

total_variance = np.sum(eig_values)
explained_2 = np.sum(eig_values[:2]) / total_variance
explained_3 = np.sum(eig_values[:3]) / total_variance
loss_2 = 1 - explained_2
loss_3 = 1 - explained_3

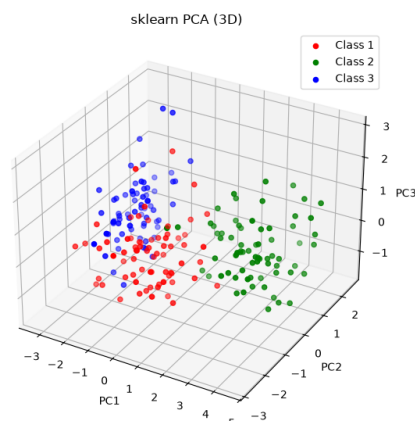
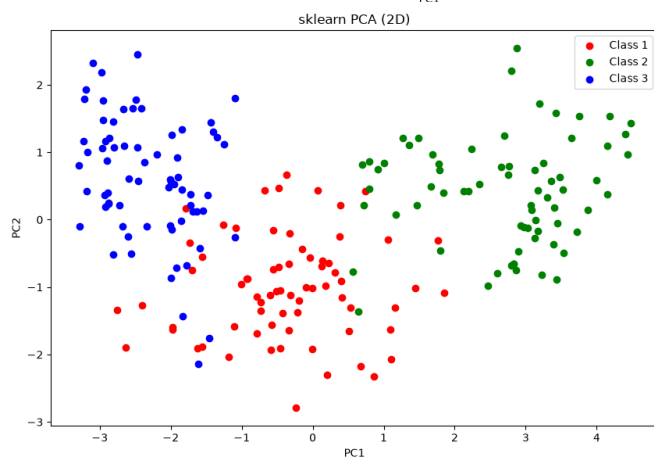
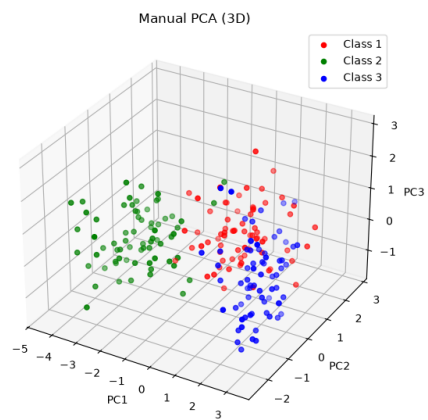
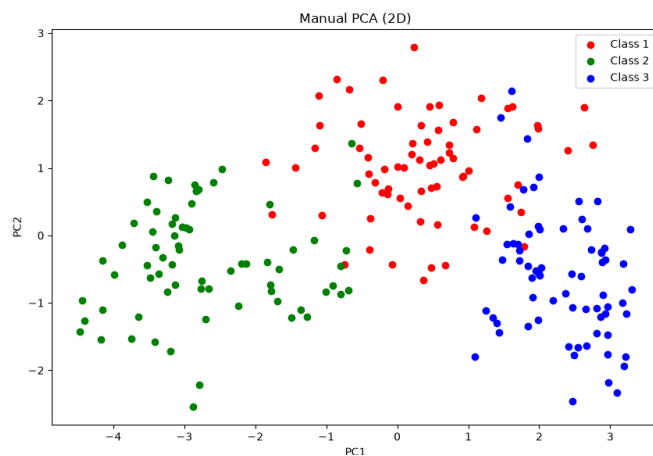
print("\nСобственные значения (ручной метод):", eig_values)
print("Собственные значения sklearn (explained_variance_, 3 комп):",
pca3.explained_variance_)

print("\nДоля объясненной дисперсии (2 компоненты, ручной метод):",
explained_2)
print("Доля объясненной дисперсии (2 компоненты, sklearn):",
pca2.explained_variance_ratio_.sum())

print("\nДоля объясненной дисперсии (3 компоненты, ручной метод):",
explained_3)
print("Доля объясненной дисперсии (3 компоненты, sklearn):",
pca3.explained_variance_ratio_.sum())

print("\nПотери информации при проекции на 2 компоненты:", loss_2)
print("Потери информации при проекции на 3 компоненты:", loss_3)

```



Пропущенные значения по столбцам до обработки:

```
area          0
perimeter     0
compactness   0
length        0
width         0
asymmetry     0
groove_length 0
class         0
dtype: int64
```

Собственные значения (ручной метод): [5.05527392e+00 1.20330286e+00 6.81247474e-01 6.86915798e-02 1.88031478e-02 5.35755786e-03 8.16283865e-04]

Собственные значения sklearn (explained_variance_, 3 комп): [5.05527392 1.20330286 0.68124747]

Доля объясненной дисперсии (2 компоненты, ручной метод): 0.8898248618491235

Доля объясненной дисперсии (2 компоненты, sklearn): 0.8898248618491231

Доля объясненной дисперсии (3 компоненты, ручной метод): 0.9866824959320444

Доля объясненной дисперсии (3 компоненты, sklearn): 0.986682495932044

Потери информации при проекции на 2 компоненты: 0.11017513815087654

Потери информации при проекции на 3 компоненты: 0.013317504067955621

Вывод: научилась применять метод PCA для осуществления визуализации данных.